

Partition to Sequencing Within Intervals: an executable reduction with output recovery

Working manuscript — reconstruction reviewed; awaiting expert review
2026-09-21

Abstract

Non-preemptive single-machine sequencing with release times and deadlines is NP-complete, and the classical proof reduces Partition to it with one task per element and a single unit-length *enforcer* whose window pins it to the middle of the horizon. An implementation of this rule was removed from the upstream Problem-Reductions library after an audit showed it unsound: with the enforcer released at $\lfloor B/2 \rfloor$, odd totals leave the enforcer schedulable, and 28 of about 40 tested infeasible instances became feasible. We give a complete, executable rule for the same endpoints. The forward map emits one task with window $[0, B + 1]$ and length s_i per element, plus a unit enforcer released at $\lceil B/2 \rceil$ with deadline $\lceil (B + 1)/2 \rceil$; recovery returns the element tasks that start before the enforcer. We prove that the construction is legal, that the enforcer is pinned exactly when the total is even and unschedulable when it is odd, and that recovery yields a valid source witness for *every* valid target output, including schedules unrelated to the one the gadget suggests. Both maps run in polynomial time and produce polynomially many bits. The classification is the known textbook result; the contribution is the missing executable rule together with the recovery obligation for all valid outputs, a repaired construction whose defect is documented, and finite evidence that includes infeasible instances, the case the removed implementation lacked.

1. Introduction

A computational problem is a pair $\Pi = (I, S)$: I is the set of legal instances and $S(x)$ is the set of valid outputs for $x \in I$. A reduction from Π_A to Π_B is a pair of deterministic polynomial-time maps: F sends a legal source instance to a legal target instance, and G sends the source instance together with *any* valid target output to a valid source output, so that $G(x, y) \in S_{A(x)}$ for every $x \in I_A$ and every $y \in S_{B(F(x))}$.

We study the pair

- **source**: Partition — given positive integers, return a subcollection summing to half the total, or `no_solution`;
- **target**: Sequencing Within Intervals — given tasks with a release time, a positive length and a deadline, return non-overlapping start times meeting every window on one machine, or `no_solution`.

The target is the classical problem $1|r_j|L_{\max} \leq 0$ without preemption, known to be NP-complete; no preemption is essential, since the preemptive variant is solvable in polynomial time. The known classification makes the pair interesting only as a **rule**: the target is a natural arithmetic scheduling benchmark whose hardness currently rests on a construction that the library does not have.

That gap is concrete rather than hypothetical. The upstream library carried an implementation of this reduction and removed it in a repair release: the audit in issue 1006 first listed the rule as sound, then corrected it to **unsound**, because the enforcer was released at $\lfloor S/2 \rfloor$ with deadline $\lceil (S + 1)/2 \rceil$, which for odd S leaves the enforcer schedulable and splits the horizon into unequal blocks of length $(S - 1)/2$ and $(S + 1)/2$. Infeasible source instances then mapped to feasible targets — 28 of about 40 sampled infeasible instances (seed 42). The accompanying pull request removed the edge and retained the model, and the rule issue was reopened for a correct re-implementation from the literature.

This paper supplies that rule. Our contributions are:

1. a construction that relocates the enforcer's release time to $\lceil B/2 \rceil$, derived here from the quoted textbook assertions rather than copied, and proved to pin the enforcer for even totals and to be unschedulable for odd ones (Theorem 2, Theorem 3);
2. the **recovery** obligation that the textbook decision proof does not state: a proof that the pre-enforcer tasks form a witness of sum $B/2$ for every valid schedule, not only for the schedule the gadget intends (Theorem 4), together with executable recovery;
3. polynomial time and encoding-size bounds for both maps (Section 5);
4. an executable, deterministic, stateless implementation with finite evidence that includes infeasible instances for both parities of the total, the case the removed implementation missed (Appendix A).

Main theorem. For the endpoints and encodings of Theorem 2, there are deterministic maps F and G with $F(x) \in I_B$ for every $x \in I_A$ and $G(x, y) \in S_{A(x)}$ for every $y \in S_{B(F(x))}$; $G(x, \text{no_solution}) = \text{no_solution}$ holds exactly when $S_{B(F(x))} = \emptyset$. Both maps run in polynomial time and their outputs have polynomially many bits. The classification itself, NP-completeness of the target, is the known textbook result; this statement concerns the rule.

The rest of the paper defines the endpoints and encodings, gives the rule, proves correctness and bounds, and records the finite evidence separately in the appendix.

2. Preliminaries

Source. The set I_A contains the finite families of positive integers $x = (s_1, \dots, s_n)$ with $n \geq 1$ and every $s_i \geq 1$, encoded as a list of binary integers. Write $B = \sum_{i=1}^n s_i$. The valid outputs $S_{A(x)}$ are the subcollections $A' \subseteq \{1, \dots, n\}$ with $\sum_{i \in A'} s_i = B/2$, encoded as strictly increasing index lists, together with the single answer `no_solution` when no such subcollection exists. A negative answer is a semantic answer, not an execution failure.

Target. The set I_B contains the finite task sets $T = \{(r_j, l_j, d_j)\}_{j=1}^m$ with integer $r_j \geq 0$, integer $l_j \geq 1$ and integer $d_j \geq 1$, encoded as a list of records. A task with $d_j < r_j + l_j$ is a legal instance that happens to be infeasible, and must be accepted as input. A schedule is a vector $\sigma \in \mathbb{N}^m$; it is *valid* when for every j

$$\sigma_j \geq r_j, \quad \sigma_j + l_j \leq d_j, \quad \text{and} \quad \forall k \neq j : \sigma_j + l_j \leq \sigma_k \quad \text{or} \quad \sigma_k + l_k \leq \sigma_j.$$

The last condition says that each task occupies the contiguous interval $[\sigma_j, \sigma_j + l_j)$ and no two tasks overlap: **preemption is forbidden**. The valid outputs $S_{B(T)}$ are the valid schedules, encoded as start-time lists, together with `no_solution` when no valid schedule exists.

Validity predicates. The definitions above are the predicates used throughout: a purported source output is accepted only if its indices are strictly increasing and in range and its sizes sum to $B/2$; a purported target output is accepted only if every window and every pairwise non-overlap condition holds.

3. Construction

Fix a legal source instance $x = (s_1, \dots, s_n)$ and let $B = \sum_i s_i$. The forward map F emits $m = n + 1$ tasks:

- for each element i , one task t_i with $r_i = 0$, $l_i = s_i$ and $d_i = B + 1$;
- one *enforcer* task t_e with $r_e = \lceil B/2 \rceil$, $l_e = 1$ and $d_e = \lceil (B + 1)/2 \rceil$.

The recovery map reads the enforcer's start time and returns the elements that begin before it:

$$G(x, \sigma) = \{i \in \{1, \dots, n\} : \sigma_i < \sigma_e\}, \quad G(x, \text{no_solution}) = \text{no_solution}.$$

The enforcer is the task appended by F , so its position $e = n + 1$ is known to G from the source instance alone; recovery needs no state from the forward run. Both maps are deterministic and stateless.

element tasks, total length $B/2$	enforcer, length 1	element tasks, total length $B/2$
-----------------------------------	--------------------	-----------------------------------

Figure 1: Even total B : the enforcer's window forces $\sigma_e = B/2$, and the element tasks must fill both remaining blocks exactly, so the elements starting before the enforcer sum to $B/2$. The bar is schematic — the blocks are equal in length and the enforcer occupies one unit — and it is drawn from the construction rather than to scale.

Figure 1 shows the even case. For odd B the enforcer is released exactly at its deadline, so no valid schedule exists at all; that is the case the removed implementation mishandled.

4. Correctness

We fix a legal source instance $x = (s_1, \dots, s_n)$, write $B = \sum_i s_i$, and let $T = F(x)$ with the enforcer index $e = n + 1$.

Theorem 1 (legal construction). Every task of T has integer $r_j \geq 0$, integer $l_j \geq 1$ and integer $d_j \geq 1$, so $T \in I_B$.

Proof. Element tasks inherit $r_i = 0$, $l_i = s_i \geq 1$ and $d_i = B + 1 \geq 2$. The enforcer has $l_e = 1$, $r_e = \lceil B/2 \rceil \geq 1$ and $d_e = \lceil (B + 1)/2 \rceil \geq 1$. All values are integers. $\square$

Theorem 2 (the enforcer is pinned exactly for even totals). Let σ be a valid schedule of T . If $B = 2m$ then $\sigma_e = m$. If $B = 2m + 1$ then no valid schedule exists.

Proof. The enforcer's own constraints are $\sigma_e \geq \lceil B/2 \rceil$ and $\sigma_e + 1 \leq \lceil (B + 1)/2 \rceil$. For $B = 2m$ these read $\sigma_e \geq m$ and $\sigma_e \leq m$, so $\sigma_e = m$. For $B = 2m + 1$ they read $\sigma_e \geq m + 1$ and $\sigma_e \leq m$, which is unsatisfiable. $\square$

Theorem 3 (the even case fills both blocks exactly). Let $B = 2m$ and let σ be a valid schedule of T . Then every element task lies entirely in $[0, m)$ or entirely in $[m + 1, 2m + 1]$, and

$$\sum_{i \in W} s_i = m = B/2$$

for $W = \{i : \sigma_i < m\}$.

Proof. By Theorem 2, $\sigma_e = m$, so the enforcer occupies $[m, m + 1)$. No element task overlaps it, so each element interval lies inside $[0, m) \cup [m + 1, 2m + 1]$; in particular no element spans the enforcer. Every element window is $[0, B + 1] = [0, 2m + 1]$, whose part outside the enforcer has measure $2m$, and the element tasks are pairwise disjoint with total length $\sum_i s_i = B = 2m$, equal to that measure.

A disjoint family contained in a measurable set, whose total measure equals the set's measure, covers the set up to a null set. Here every start and length is an integer, so every task interval has integer endpoints; the complement of their finite union inside $[0, m) \cup [m + 1, 2m + 1]$ is then a finite union of intervals with integer endpoints, and any non-empty such complement has measure at least 1. The complement has measure 0, so it is empty, and the elements occupy both blocks exactly. The elements starting before m therefore have total length m , that is $\sum_{i \in W} s_i = m$. $\square$

Theorem 4 (correctness of the rule). For every legal source instance x and every valid target output $y \in S_{B(F(x))}$, the recovered value $G(x, y)$ lies in $S_{A(x)}$.

Proof. Let $y \in S_{B(F(x))}$ be valid.

First, $y = \text{no_solution}$. Then $S_{B(F(x))} = \emptyset$. Suppose x had a witness A' with $\sum_{i \in A'} s_i = B/2$. Then $B = 2m$ is even. Schedule the tasks of A' contiguously in increasing index order starting at time 0, the enforcer at m , and the remaining element tasks contiguously after it, ending at $2m + 1$. Every element ends by $2m + 1 = B + 1 = d_i$ and starts no earlier than $0 = r_i$; the enforcer lies in $[m, m + 1)$, inside its window by the computation in Theorem 2; and no two tasks overlap. So $S_{B(F(x))} \neq \emptyset$, a contradiction. Hence $S_{A(x)} = \emptyset$ and $G(x, y) = \text{no_solution} \in S_{A(x)}$.

Second, $y = \sigma$ is a schedule. By Theorem 2, $B = 2m$ is even and $\sigma_e = m$. By Theorem 3 the set $W = \{i : \sigma_i < \sigma_e\}$ satisfies $\sum_{i \in W} s_i = m = B/2$, so $W \in S_{A(x)}$, and $G(x, y) = W$. $\square$

The two directions also show that the negative answer is not vacuous: a witness of x always yields a valid schedule of $F(x)$, by the construction used in the first case. Hence $S_{B(F(x))}$ is empty exactly when $S_{A(x)}$ is.

5. Complexity

Write $|x|$ for the length of the source encoding, so $|x| = \Theta(n + \log B)$. The forward map performs $O(n)$ integer operations on numbers of bit length $O(\log B)$ and emits $n + 1$ tasks: its running time is $O(n \log B)$ and its output has $O(n \log B)$ bits, both polynomial in $|x|$.

Recovery compares the n element start times with σ_e , costing $O(n \log B)$ bit operations because each comparison touches numbers of bit length $O(\log B)$. Its output is a list of at most n indices, each of bit length $\Theta(\log n)$, so it has $\Theta(n \log n)$ bits in the worst case, attained by the subset of all indices. Both maps are therefore polynomial in $|x|$ and, for recovery, in $|x| + |y|$. No randomness, solver call or shared state is involved.

6. Conclusion

We gave a complete rule for Partition to Sequencing Within Intervals: a corrected enforcer construction, executable recovery proved correct for every valid target output, and polynomial bounds for both maps. On odd totals the enforcer is released at its deadline and the target is infeasible, so the defect that removed the previous implementation — infeasible sources mapping to feasible targets — is repaired rather than patched.

The classification is not new. The theorem proved here is the textbook NP-completeness of the target, and the paper claims no new complexity result; its contribution is the rule itself, the recovery obligation for all valid outputs, and finite evidence that covers infeasible instances. Three limits are worth stating. First, the construction is a reconstruction: the primary source text was not available here, and the enforcer window is derived from the assertions quoted upstream, which are consistent only under the rounding convention $\lceil x \rceil = \lceil x \rceil$. Second, for odd totals the enforcer has $r_e = d_e$, an empty window; that is legal under the definitions used here and under the current upstream model, but a released model that requires $r_j + l_j \leq d_j$ cannot represent it, and registering the rule there would need an explicitly infeasible target built from satisfiable windows instead. Third, G is defined on valid target outputs; no promise is made for malformed schedules.

References

1. M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*, W. H. Freeman, 1979. Chapter 3, Theorem 3.8, and problem [SS1] **Sequencing Within Intervals**; the theorem number, page and bracket notation are taken from the upstream quotation referenced below, since the volume itself was not available while writing.
2. Problem-Reductions, ground-truth reduction issues: **[Rule] Partition to Sequencing Within Intervals**, issue 205, <https://github.com/CodingThrust/problem-reductions/issues/205> (open; quoted textbook proof and the requested deliverable).
3. Problem-Reductions, audit and repair records: issue 1006 (<https://github.com/CodingThrust/problem-reductions/issues/1006>) and pull request 1052 (<https://github.com/CodingThrust/problem-reductions/pull/1052>), which removed the unsound implementation and retained the model.
4. The reduction contract adopted by this campaign, `research/reduction.md`, which states the two-map obligation and cites R. Fenner, L. Fortnow, S. Naik and J. Rogers, **Complements of Multivalued Functions**, for the multivalued-function formulation of metric many-one reducibility.

A. Verification and reproducibility

Finite evidence is reported separately from the proof above; no finite test establishes the theorem.

Prerequisites. The campaign’s capability probe records CPython 3.12.11 through uv with a committed lockfile, OR-Tools 9.15.6755, and Typst 0.15.1 for this document. The environment is reproduced with `uv sync --locked`; no solver, subprocess, shell or wall-clock timeout appears in any check.

Endpoints and encodings. Source instances, target instances and both output sets are the JSON encodings of `work/contract.md`, fixed before the candidate was written. A candidate is executed only through its command `contract: python3 algorithm.py` reads one source instance and writes one target instance; `python3 algorithm.py --extract` reads source and `target_solution` and writes one source output.

Testing foundation. `work/check.py --self-test` decides both endpoints with two independent methods — subset-sum dynamic programming cross-checked against exhaustive enumeration for the source, and a subset dynamic program cross-checked against a CP-SAT `AddNoOverlap` encoding for the target — validates every output against the definitions, and rejects 18 deliberately malformed fixtures (wrong sums, duplicate or out-of-range indices, both or neither output keys, overlapping tasks, a task past its deadline, a negative start, a schedule of the wrong length). Sixteen injected source instances carry hand-checkable ground truth, and infeasible instances cover both parities of the total.

Results. The prepared suite (`work/check.py --candidate`) passed on 16 instances with 59 distinct valid target outputs fed through recovery. The independent verifier (`work/verify.py`), which imports neither the checker nor the candidate and uses different algorithms for both endpoints, passed on 52 instances — 12 degenerate or extreme cases (single items, values up to 10^{12} , ten unit items, powers of two, consecutive integers, large coprime values) and 40 deterministic pseudo-random instances (seed 20260921, 1–7 items, sizes 1–12) — covering 153 distinct valid target outputs: 113 schedules and 40 `no_solution` answers. Target feasibility matched source existence in every case. Finite bounds: at most 12 distinct target outputs per constructed instance, source sizes up to $n = 10$, values up to 10^{12} ; feasibility above eight tasks rests on the solver alone. All outputs are retained in the campaign repository.

Independent review. A fresh-context reviewer audited the construction, the proof and the implementation with its own checker, enumerating all valid target outputs on 2 721 small instances (378 670 schedules, every one with the enforcer pinned and the pre-enforcer set summing to $B/2$), and decided the result eligible for expert review. Its four findings — an integrality premise in Theorem 3, the recovery bit-size bound, the attribution of the defective window, and the empty-window integration caveat — are repaired or recorded in the campaign records, and a focused follow-up review confirmed the repairs and verified that the candidate, tests and evidence were unchanged.